FLUTTER UI TOOLKIT

Flutter Widgets

Building blocks of every Flutter UI: concepts, layouts, code and diagrams

OVERVIEW

Agenda

1 Widget fundamentals

Widget tree, stateless vs stateful, lifecycle, BuildContext

2 Layout widgets

Container, Row, Column, Stack, Expanded, lists and grids

3 Input and display

Text, images, buttons, forms, cards

4 App structure

Scaffold, navigation, theming

5 State and practice

State management and best practices

6 Wrap-up

Summary and resources

OVERVIEW

What is Flutter?

1 One codebase

Write Dart once, ship to Android, iOS, web, Windows, macOS and Linux.

2 Native speed

Compiled ahead of time; Flutter draws every pixel itself with its own engine.

3 Hot reload

See code changes in under a second while keeping app state.

Declarative UI: UI = f(state). You describe what the screen should look like for the current state; Flutter figures out how to update it.

FUNDAMENTALS

Everything is a widget

V Visible

Text, Icon, Image, Button

L Layout

Row, Column, Stack, Padding

I Interactive

GestureDetector, TextField, Switch

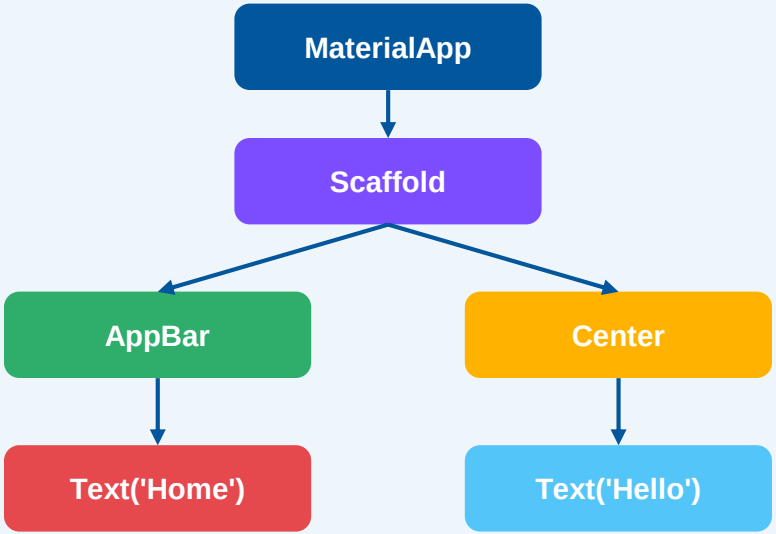
A App-level

MaterialApp, Theme, Navigator

- Widgets are immutable, lightweight descriptions of UI, cheap to recreate on every rebuild
- Small widgets compose into bigger ones: the whole app is one widget tree
- Prefer composition over inheritance

FUNDAMENTALS

The widget tree



Parents contain children; data flows down, events bubble up.

main.dart

```dart
void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Home'),
        ),
        body: const Center(
          child: Text('Hello'),
        ),
      ),
    );
  }
}
```

FUNDAMENTALS

Three trees behind the scenes



Immutable configuration you write. Rebuilt often and cheap.

Long-lived instances that link widgets to the screen and hold State.

Does layout, painting and hit testing. Expensive, so reused.

Why it matters

- When a widget rebuilds, Flutter compares it to the old one and only updates RenderObjects that changed
- Keys help Flutter match elements when widgets move inside a list

FUNDAMENTALS

StatelessWidget

- No mutable state; output depends only on constructor arguments
- Use for static or parent-driven UI
- Add const constructors so Flutter can skip rebuilding identical widgets
- Mark every field final

profile_badge.dart

```
class ProfileBadge extends StatelessWidget {
  const ProfileBadge({
    super.key,
    required this.name,
    required this.role,
  });

  final String name;
  final String role;

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        CircleAvatar(child: Text(name[0])),
        const SizedBox(height: 8),
        Text(name, style: const TextStyle(
            fontWeight: FontWeight.bold)),
        Text(role),
      ],
    );
  }
}
```

FUNDAMENTALS

StatefulWidget

- The widget is immutable; its State object lives on
- setState() marks the widget dirty so build() runs again
- Use for counters, forms, toggles and animations
- Underscore prefix makes State private to the file

counter.dart

```
class Counter extends StatefulWidget {
  const Counter({super.key});

  @override
  State<Counter> createState() =>
      _CounterState();
}

class _CounterState extends State<Counter> {
  int _count = 0;

  void _increment() {
    setState(() => _count++);
  }

  @override
  Widget build(BuildContext context) {
    return FilledButton(
      onPressed: _increment,
      child: Text('Taps: ' + _count.toString()),
    );
  }
}
```

FUNDAMENTALS

Stateless vs Stateful

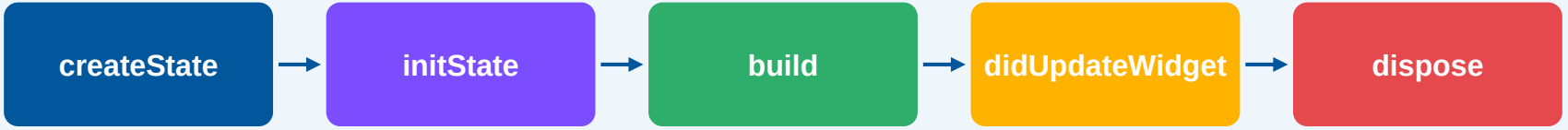
StatelessWidget

- Immutable inputs only
- One class, one build()
- Rebuilds when parent passes new data
- Labels, icons, layout wrappers

StatefulWidget

- Owns mutable State object
- Two classes: widget + State
- Rebuilds on setState() too
- Forms, toggles, animations

State lifecycle



Framework creates the State

One-time setup

Runs on every rebuild

Parent gave new config

Release resources

lifecycle.dart

```dart
class _TimerState extends State<TimerBox> {
  late final Timer _timer;
  int _seconds = 0;

  @override
  void initState() {
    super.initState();
    _timer = Timer.periodic(
      const Duration(seconds: 1),
      (_) => setState(() => _seconds++),
    );
  }
```

BuildContext

- A handle to this widget's position in the tree
- Used to look up ancestors: Theme, MediaQuery, Navigator
- Each widget gets its own context
- After an await, check context.mounted before using it

context_demo.dart

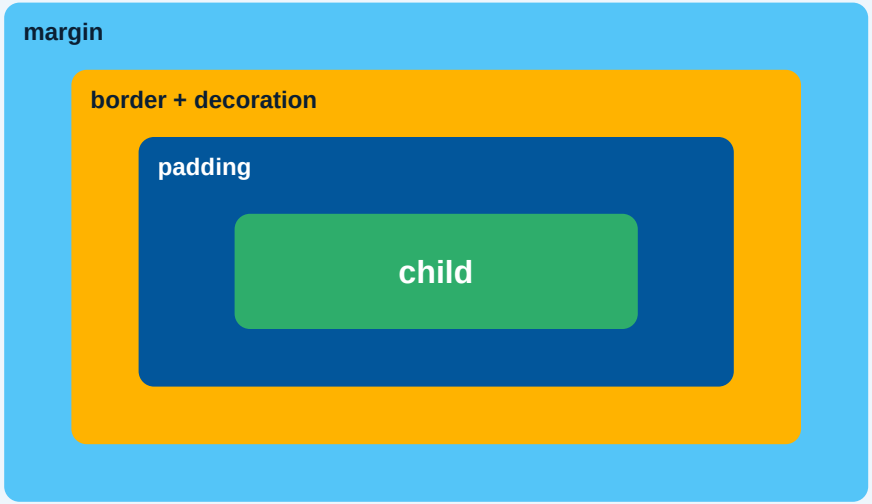
```
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  final size = MediaQuery.of(context).size;
  final isWide = size.width > 600;

  return Container(
    color: theme.colorScheme.primary,
    padding: EdgeInsets.all(isWide ? 32 : 16),
    child: Text(
      'Width: ' + size.width.round().toString(),
      style: theme.textTheme.titleLarge,
    ),
  );
}

Future<void> save() async {
  await repo.save();
  if (!context.mounted) return;
  Navigator.pop(context);
}
```

LAYOUT

Container and the box model



- Container combines sizing, padding, margin, color and decoration
- Use SizedBox, Padding or DecoratedBox when you need only one of these

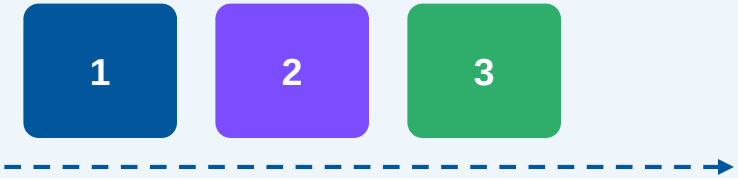
container.dart

```
Container(
  width: 220,
  margin: const EdgeInsets.all(12),
  padding: const EdgeInsets.symmetric(
    horizontal: 20, vertical: 14),
  decoration: BoxDecoration(
    color: Colors.blue.shade50,
    borderRadius: BorderRadius.circular(16),
    border: Border.all(
      color: Colors.blue, width: 2),
    boxShadow: const [
      BoxShadow(
        color: Colors.black26,
        blurRadius: 8,
        offset: Offset(0, 4)),
    ],
  ),
  child: const Text('Hello Flutter'),
)
```

LAYOUT

Row and Column

Row (main axis: horizontal)



Column (main axis: vertical)



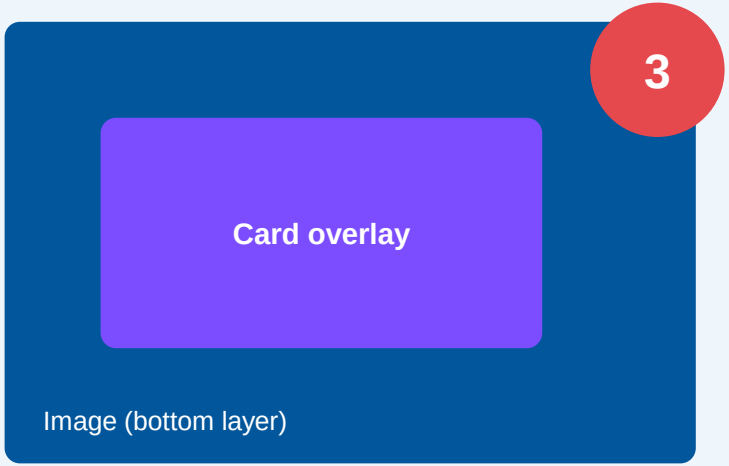
- mainAxisAlignment: start, center, end, spaceBetween, spaceEvenly
- crossAxisAlignment: start, center, end, stretch

row_column.dart

```
Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    const Text('Settings',
        style: TextStyle(fontSize: 20)),
    const SizedBox(height: 12),
    Row(
      mainAxisAlignment:
          MainAxisAlignment.spaceBetween,
      children: [
        const Text('Notifications'),
        Switch(
          value: true,
          onChanged: (v) {},
        ),
      ],
    ),
  ],
)
```

LAYOUT

Stack and Positioned



- Children paint in order; the last is on top
- Positioned pins a child to edges
- Ideal for badges, captions and overlays

stack.dart

```
Stack(
  clipBehavior: Clip.none,
  children: [
    ClipRRect(
      borderRadius: BorderRadius.circular(16),
      child: Image.asset('assets/hero.jpg',
          height: 180, fit: BoxFit.cover),
    ),
    const Positioned(
      left: 16,
      bottom: 16,
      child: Text('Mountain trip',
        style: TextStyle(color: Colors.white,
            fontSize: 20)),
    ),
    Positioned(
      top: -8, right: -8,
      child: Badge(label: Text('3')),
    ),
  ],
)
```

LAYOUT

Expanded and Flexible



Row width is split 1 : 2 : 1 across the free space

E Expanded

Fills its share of free space (tight fit)

F Flexible

May be smaller than its share (loose fit)

expanded.dart

```
Row(
  children: [
    Expanded(
      flex: 1,
      child: Container(color: Colors.blue),
    ),
    Expanded(
      flex: 2,
      child: Container(color: Colors.purple),
    ),
    Flexible(
      child: Container(
        color: Colors.green,
        child: const Text('Only as wide\nas needed'),
      ),
    ),
  ],
)
```

LAYOUT

Padding, Align, SizedBox, Spacer

P **Padding**

Insets around a child with EdgeInsets

A **Center / Align**

Positions a child inside its parent

S **SizedBox**

Fixed width or height; common as a gap

Sp **Spacer**

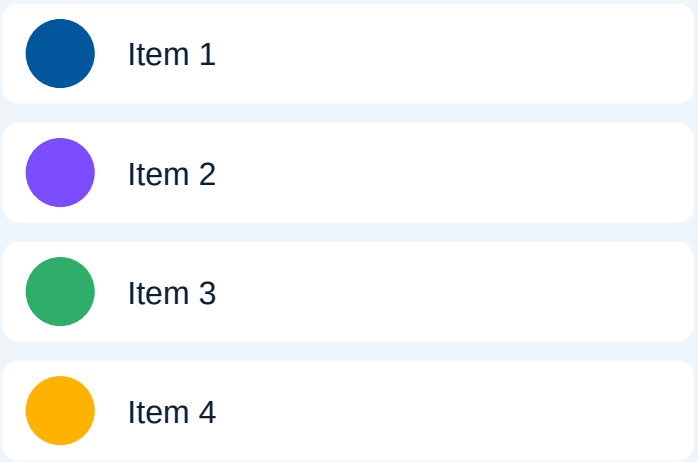
Flexible empty space in Row or Column

spacing.dart

```
Padding(
  padding: const EdgeInsets.all(16),
  child: Column(
    children: [
      const Align(
        alignment: Alignment.centerLeft,
        child: Text('Total'),
      ),
      const SizedBox(height: 12),
      Row(
        children: const [
          Text('Items'),
          Spacer(),
          Text('42'),
        ],
      ),
    ],
  ),
)
```

LAYOUT

ListView



- builder creates rows lazily as they scroll into view
- separated adds dividers between rows
- Set shrinkWrap only for short lists
- Use ScrollController for scroll position

list_view.dart

```dart
ListView.separated(
  padding: const EdgeInsets.all(16),
  itemCount: users.length,
  separatorBuilder: (_, __) =>
      const Divider(height: 1),
  itemBuilder: (context, index) {
    final user = users[index];
    return ListTile(
      leading: CircleAvatar(
        child: Text(user.name[0]),
      ),
      title: Text(user.name),
      subtitle: Text(user.email),
      onTap: () => openUser(user),
    );
  },
)
```

LAYOUT

GridView



- GridView.count: fixed column count
- GridView.extent: max tile width
- GridView.builder: lazy grids for large data
- childAspectRatio shapes each tile

grid_view.dart

```
GridView.builder(
  padding: const EdgeInsets.all(12),
  gridDelegate:
      const
SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 3,
    mainAxisSpacing: 8,
    crossAxisSpacing: 8,
    childAspectRatio: 1,
  ),
  itemCount: photos.length,
  itemBuilder: (context, i) => ClipRRect(
    borderRadius: BorderRadius.circular(12),
    child: Image.network(photos[i],
        fit: BoxFit.cover),
  ),
)
```

How layout works: constraints

Constraints go down → **Sizes go up** → **Parent positions**

Parent tells child min and max width and height

Child picks a size inside those constraints

Parent decides where the child sits

constraints.dart

```
// The child cannot exceed 200 wide:
ConstrainedBox(
  constraints: const BoxConstraints(maxWidth: 200),
  child: Container(color: Colors.blue, height: 80),
)

// LayoutBuilder reads parent constraints:
LayoutBuilder(
  builder: (context, box) {
    return box.maxWidth > 600
        ? const WideLayout()
```

DISPLAY

Text and Icon

Flutter

Styled with TextStyle

- TextStyle sets font, size, weight, color and spacing
- Text.rich mixes styles with TextSpan
- maxLines and overflow handle long text
- Icon draws glyphs from Icons or custom fonts

text.dart

```
Text.rich(
  TextSpan(
    text: 'Hello ',
    style: Theme.of(context).textTheme.titleLarge,
    children: const [
      TextSpan(
        text: 'Flutter',
        style: TextStyle(
          fontWeight: FontWeight.bold,
          color: Colors.blue,
        ),
      ),
    ],
  ),
  maxLines: 2,
  overflow: TextOverflow.ellipsis,
),
const Icon(Icons.favorite,
    color: Colors.red, size: 28)
```

DISPLAY

Images

A Image.asset

Bundled; declare in pubspec.yaml

N Image.network

Remote URL with loading and error builders

F BoxFit

cover, contain, fill, fitWidth control scaling

image.dart

```
Image.network(
  'https://example.com/photo.jpg',
  width: 240,
  height: 160,
  fit: BoxFit.cover,
  semanticLabel: 'Mountain view',
  loadingBuilder: (c, child, p) {
    if (p == null) return child;
    return const Center(
      child: CircularProgressIndicator(),
    );
  },
  errorBuilder: (c, e, s) =>
      const Icon(Icons.broken_image),
)

# pubspec.yaml
flutter:
  assets:
    - assets/images/
```

DISPLAY

Buttons





FAB

- onPressed null disables the button
- Use IconButton for icon-only actions
- Style with styleFrom

buttons.dart

```
Row(
  children: [
    ElevatedButton.icon(
      onPressed: () => save(),
      icon: const Icon(Icons.save),
      label: const Text('Save'),
    ),
    const SizedBox(width: 8),
    OutlinedButton(
      onPressed: null, // disabled
      child: const Text('Cancel'),
    ),
    TextButton(
      style: TextButton.styleFrom(
        foregroundColor: Colors.red),
      onPressed: () {},
      child: const Text('Delete'),
    ),
  ],
)
```

INPUT

TextField and Forms

Email

Required field

- Form + GlobalKey validates many fields at once
- TextEditingController reads and sets text
- Dispose controllers in dispose()

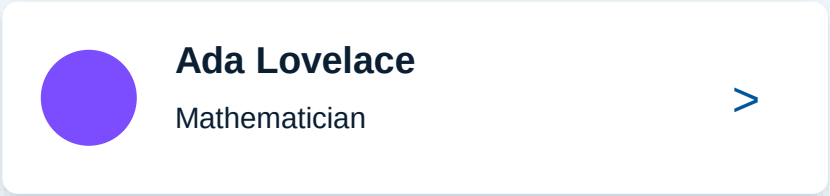
login_form.dart

```
final _formKey = GlobalKey<FormState>();
final _email = TextEditingController();

Form(
  key: _formKey,
  child: Column(children: [
    TextFormField(
      controller: _email,
      keyboardType: TextInputType.emailAddress,
      decoration: const InputDecoration(
        labelText: 'Email',
        border: OutlineInputBorder(),
      ),
      validator: (v) =>
          (v == null || !v.contains('@'))
              ? 'Enter a valid email' : null,
    ),
    FilledButton(
      onPressed: () {
        if (_formKey.currentState!.validate()) {}
      },
      child: const Text('Sign in'),
    ),
  ]),
)
```

DISPLAY

Card and ListTile



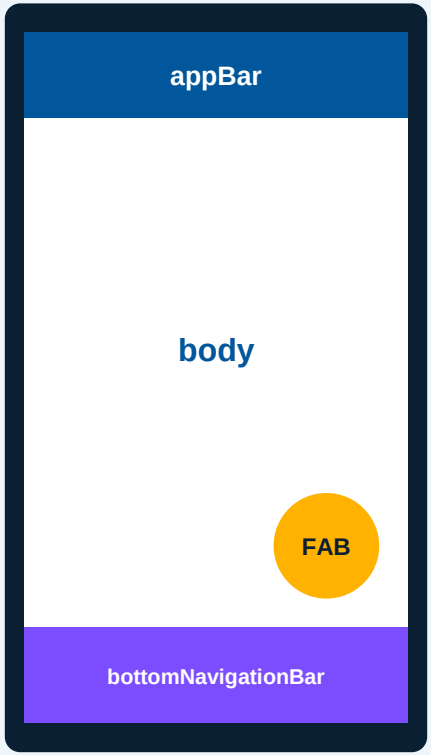
- Card gives elevation, rounded corners and surface color
- ListTile: leading, title, subtitle, trailing
- InkWell adds ripple on tap
- Card + ListTile is the core of most list screens

contact_card.dart

```
Card(
  elevation: 3,
  margin: const EdgeInsets.all(12),
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(16),
  ),
  child: ListTile(
    leading: const CircleAvatar(
      child: Icon(Icons.person),
    ),
    title: const Text('Ada Lovelace'),
    subtitle: const Text('Mathematician'),
    trailing: const Icon(Icons.chevron_right),
    onTap: () => Navigator.pushNamed(
        context, '/profile'),
  ),
)
```

APP STRUCTURE

Scaffold anatomy



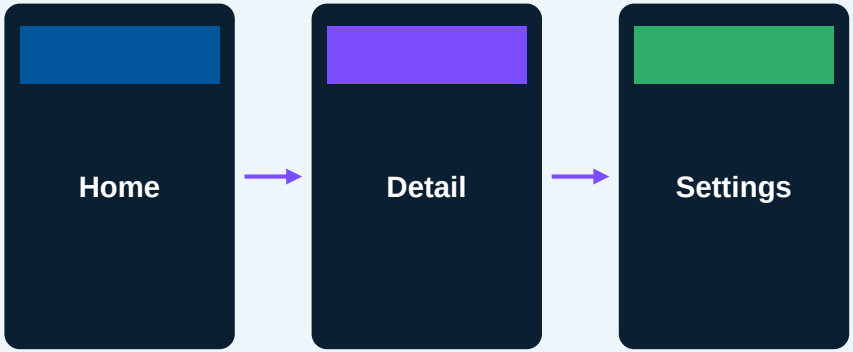
- Scaffold provides the basic Material page
- Also: drawer, snackbars, bottom sheets
- SafeArea avoids notches

home_page.dart

```
Scaffold(
  appBar: AppBar(
    title: const Text('Home'),
    actions: [
      IconButton(
        icon: const Icon(Icons.search),
        onPressed: () {},
      ),
    ],
  ),
  body: const SafeArea(
    child: Center(child: Text('Content')),
  ),
  floatingActionButton: FloatingActionButton(
    onPressed: () {},
    child: const Icon(Icons.add),
  ),
  bottomNavigationBar: NavigationBar(
    destinations: const [
      NavigationDestination(
        icon: Icon(Icons.home), label: 'Home'),
      NavigationDestination(
        icon: Icon(Icons.person), label: 'Me'),
    ],
  ),
)
```

APP STRUCTURE

Navigation



push() adds a route on top of the stack; pop() returns to the previous one

- Navigator 1.0: push, pop, named routes
- go_router: declarative routing and deep links
- Return data with pop(result)

navigation.dart

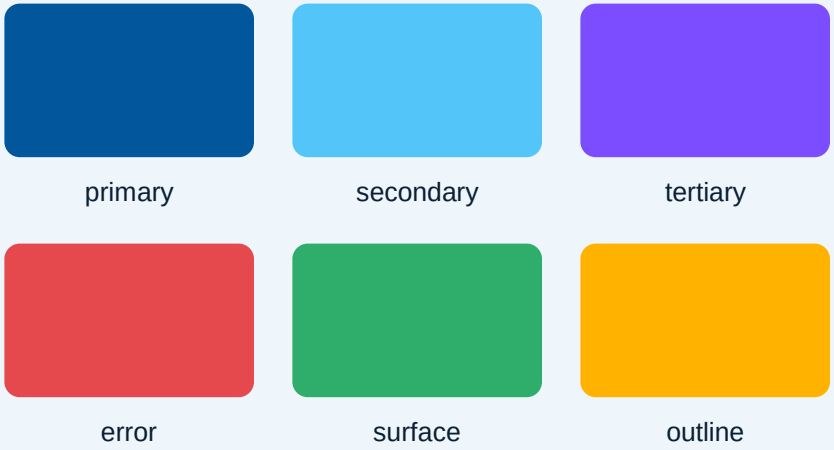
```
// Push and await a result
final choice = await Navigator.push<String>(
  context,
  MaterialPageRoute(
    builder: (_) => const DetailPage(id: 7),
  ),
);

// Named routes
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (_) => const HomePage(),
    '/settings': (_) => const SettingsPage(),
  },
);
Navigator.pushNamed(context, '/settings');

// Go back with a value
Navigator.pop(context, 'done');
```

APP STRUCTURE

Theming



- ColorScheme.fromSeed builds a full Material 3 palette
- Set theme and darkTheme, switch with themeMode
- Read tokens via Theme.of(context)

theme.dart

```
MaterialApp(
  themeMode: ThemeMode.system,
  theme: ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.indigo,
    ),
    cardTheme: CardTheme(
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16),
      ),
    ),
  ),
  darkTheme: ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.indigo,
      brightness: Brightness.dark,
    ),
  ),
  home: const HomePage(),
)
```

State management options

1 setState

Local to one widget. Simplest option.

2 InheritedWidget / Provider

Share state down the tree with minimal boilerplate.

3 Riverpod

Compile-safe providers; easy testing and caching.

4 Bloc

Events in, states out; clear structure for large teams.

provider_example.dart

```
class CartModel extends ChangeNotifier {
  final List<String> _items = [];
  List<String> get items => List.unmodifiable(_items);
  void add(String item) { _items.add(item); notifyListeners(); }
}

// Provide once, read anywhere below
ChangeNotifierProvider(create: (_) => CartModel(), child: const App());
final count = context.watch<CartModel>().items.length;
context.read<CartModel>().add('Book');
```

PRACTICE

Best practices

1 Use const

const widgets are created once and skipped on rebuild

2 Extract widgets

Small classes beat big build() methods and helper functions

3 Build lazily

ListView.builder and GridView.builder for long data

4 Keep build() pure

No network calls or heavy work inside build

5 Dispose resources

Cancel timers, close controllers and streams

6 Test widgets

Use flutter_test and the Flutter Inspector

Key takeaways

- Everything is a widget; compose small pieces
- Stateless for static UI, Stateful for local state
- Row, Column, Stack and ListView cover most layouts
- Constraints go down, sizes go up

Resources: docs.flutter.dev | dartpad.dev | Widget of the Week on YouTube

PART 2

State, Provider and go_router

setState(), the Provider package, declarative routing (Navigator 2.0) and deep linking

PART 2

Roadmap

1 setState()

How it works, rules, async safety, lifting state up, performance, ValueNotifier

2 Provider

ChangeNotifier, watch / read / select, Consumer, ProxyProvider, pitfalls

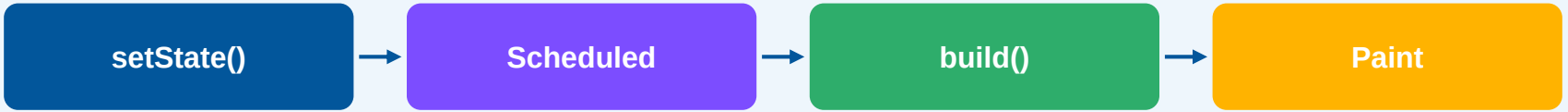
3 go_router

Navigator 2.0 ideas, params, redirects, shell routes, deep links, testing

Local state -> shared state -> URL-driven navigation

SETSTATE

What setState() does



Runs your closure, marks State dirty

Rebuild queued for next frame

New widget description created

Only changed render objects update

like_button.dart

```
class _LikeButtonState extends State<LikeButton> {
  bool _liked = false;

  void _toggle() => setState(() => _liked = !_liked);

  @override
  Widget build(BuildContext context) {
    return IconButton(
      icon: Icon(_liked ? Icons.favorite : Icons.favorite_border,
          color: _liked ? Colors.red : null),
      onPressed: _toggle,
    );
  }
}
```

Rules of setState()

Do

- Change state inside the callback
- Keep the closure small and synchronous
- Call it only from a State object
- Check mounted after any await

Don't

- Call setState inside build()
- Call it after dispose()
- Do heavy work or await inside the closure
- Mutate state without calling it

setState() with async work

- Model three UI states: loading, error, data
- Do the await outside the closure
- Check mounted before touching State after an await
- Keep the closure tiny: only assignments

load_users.dart

```
Future<void> _load() async {
  setState(() => _loading = true);
  try {
    final data = await api.fetchUsers();
    if (!mounted) return; // widget may be gone
    setState(() {
      _users = data;
      _loading = false;
    });
  } catch (e) {
    if (!mounted) return;
    setState(() {
      _error = e.toString();
      _loading = false;
    });
  }
}
```

Controls driven by setState()

- Each control reports a new value through onChanged
- setState stores it and the UI redraws
- Great for toggles, sliders, checkboxes, tabs



Dark mode

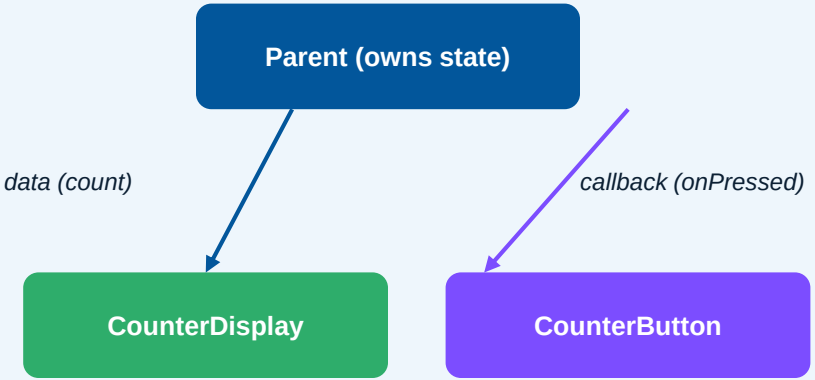


settings_panel.dart

```
bool _dark = false;
double _volume = 0.5;

Column(children: [
  SwitchListTile(
    title: const Text('Dark mode'),
    value: _dark,
    onChanged: (v) => setState(() => _dark = v),
  ),
  Slider(
    value: _volume,
    onChanged: (v) => setState(() => _volume = v),
  ),
  Text('Volume: ' +
      (_volume * 100).round().toString()),
])
```

Lifting state up



- Share state by moving it to the nearest common parent
- Pass data down, callbacks up
- Children stay stateless and reusable

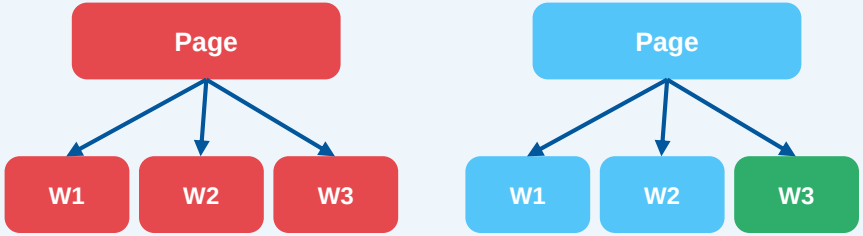
parent.dart

```
class _ParentState extends State<Parent> {
  int _count = 0;

  @override
  Widget build(BuildContext context) {
    return Column(children: [
      CounterDisplay(count: _count),
      CounterButton(
        onPressed: () => setState(() => _count++),
      ),
    ]);
  }
}

class CounterButton extends StatelessWidget {
  const CounterButton({super.key, required this.onPressed});
  final VoidCallback onPressed;
  // build: FilledButton(onPressed: onPressed, ...)
}
```

Keep rebuilds small



setState on the page **setState in a leaf**

- Move setState into the smallest widget that changes
- Use const for subtrees that never change
- Pass a prebuilt child to builders

page.dart

```
Scaffold(
  body: Column(children: [
    const HeavyHeader(), // const: skipped
    const ProductList(),
    LikeButton(), // owns tiny state
  ]),
)

// Anti-pattern: whole page is stateful
class _PageState extends State<Page> {
  bool _liked = false; // rebuilds EVERYTHING
  ...
}
```

ValueNotifier alternative

- Holds one value and notifies listeners
- ValueListenableBuilder rebuilds only its builder
- No StatefulWidget needed for tiny state
- Always dispose the notifier

value_notifier.dart

```
final _counter = ValueNotifier<int>(0);

ValueListenableBuilder<int>(
  valueListenable: _counter,
  builder: (context, value, child) {
    return Column(children: [
      Text('Count: ' + value.toString()),
      child!, // built once
    ]);
  },
  child: const Icon(Icons.timer, size: 40),
)

FilledButton(
  onPressed: () => _counter.value++,
  child: const Text('Add'),
)

// in dispose(): _counter.dispose();
```

When to outgrow setState()

1 Shared by many screens

Cart, user, settings used in several places

2 Prop drilling

Passing values through many layers

3 Logic inside widgets

Business rules mixed with UI code

4 Hard to test

State locked inside widget classes

5 Async and caching

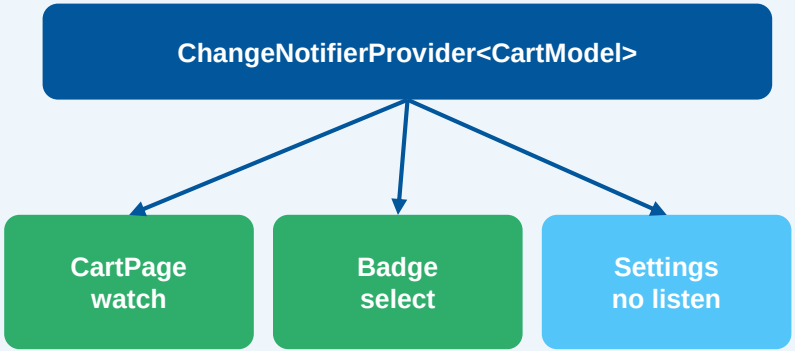
Loading, refreshing and reusing data

6 Cross-widget updates

A change here must update there

Next step: Provider

Why Provider?



- A friendly wrapper around InheritedWidget
- Creates, shares and disposes your models
- Only listening widgets rebuild
- Featured in Flutter's own state-management docs

pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.0

# Install
#   flutter pub add provider

# Import in Dart files
import 'package:provider/provider.dart';
```

PROVIDER

ChangeNotifier model

- Put state and logic in a plain Dart class
- Expose read-only getters and unmodifiable lists
- Call notifyListeners() after every change
- Keep UI code out of the model

cart_model.dart

```
class CartModel extends ChangeNotifier {
  final List<Item> _items = [];

  List<Item> get items => List.unmodifiable(_items);
  int get count => _items.length;
  double get total =>
      _items.fold(0, (sum, i) => sum + i.price);

  void add(Item item) {
    _items.add(item);
    notifyListeners();
  }

  void remove(Item item) {
    _items.remove(item);
    notifyListeners();
  }

  void clear() {
    _items.clear();
    notifyListeners();
  }
}
```

Providing models

- Place providers above the widgets that need them
- MultiProvider avoids deep nesting
- create is lazy; the model is built on first use
- ChangeNotifierProvider disposes the model for you
- Use .value to supply an existing instance

main.dart

```
void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(
          create: (_) => CartModel(),
        ),
        ChangeNotifierProvider(
          create: (_) => ThemeModel(),
        ),
        Provider(
          create: (_) => ApiService(),
        ),
      ],
      child: const MyApp(),
    ),
  );
}
```

PROVIDER

watch, read and select

W context.watch<T>()

Listens; rebuilds on any change. Use inside build().

R context.read<T>()

No listening. Use in callbacks like onPressed.

S context.select<T, R>()

Rebuilds only when the selected value changes.

cart_badge.dart

```
Widget build(BuildContext context) {
  final total = context.watch<CartModel>().total;       // rebuilds on change
  final n = context.select<CartModel, int>((c) => c.count); // only if count changes
  return IconButton(
    icon: Badge(label: Text(n.toString()), child: const Icon(Icons.shopping_cart)),
    tooltip: 'Total: ' + total.toStringAsFixed(2),
    onPressed: () => context.read<CartModel>().clear(),  // read: no listening
  );
}
```

PROVIDER

Consumer and Selector

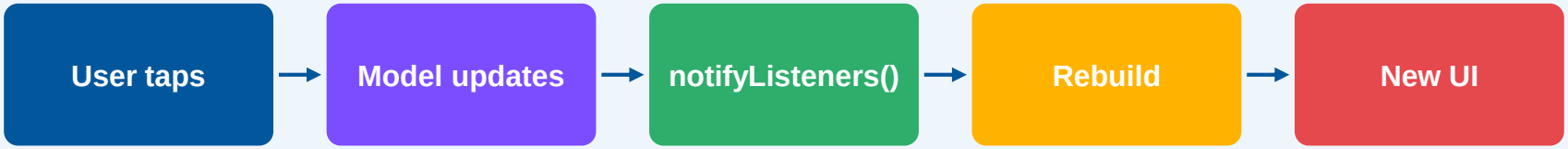
- Consumer rebuilds just its builder subtree
- Its child parameter is built once
- Selector rebuilds only when the chosen value changes
- Prefer them when build() is large

consumer.dart

```
Consumer<CartModel>(
  builder: (context, cart, child) {
    return Column(children: [
      Text('Total: ' +
          cart.total.toStringAsFixed(2)),
      child!, // static, not rebuilt
    ]);
  },
  child: const Divider(),
)

Selector<CartModel, int>(
  selector: (_, cart) => cart.count,
  builder: (_, count, __) =>
      Text('Items: ' + count.toString()),
)
```

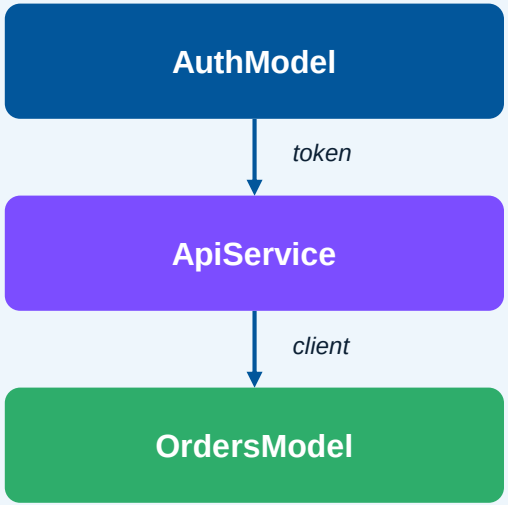
One-way data flow



Why this works well

- Widgets never change state directly; they ask the model
- The model is the single source of truth, so it is easy to test without UI

ProxyProvider: dependent models



providers.dart

```
MultiProvider(
  providers: [
    ChangeNotifierProvider(
        create: (_) => AuthModel()),
    ProxyProvider<AuthModel, ApiService>(
      update: (_, auth, previous) =>
          ApiService(token: auth.token),
    ),
    ChangeNotifierProxyProvider<ApiService,
        OrdersModel>(
      create: (_) => OrdersModel(),
      update: (_, api, orders) =>
          orders!..attach(api),
    ),
  ],
  child: const MyApp(),
)
```

- Rebuilds dependents when upstream models change
- Order matters: dependencies come first

FutureProvider and StreamProvider

- FutureProvider exposes the result of a Future
- StreamProvider exposes the latest stream value
- initialData avoids null while loading
- Read the value with context.watch<T>()

async_providers.dart

```
FutureProvider<List<Product>>(
  create: (_) => api.fetchProducts(),
  initialData: const [],
  child: const ProductsPage(),
)

StreamProvider<User?>(
  create: (_) => authService.userChanges(),
  initialData: null,
  child: const App(),
)

// In a widget
final products =
    context.watch<List<Product>>();
final user = context.watch<User?>();
```

Common pitfalls

Avoid

- context.read in build() for values that must update
- Creating the model inside build()
- Exposing a mutable list directly
- Providers placed below the widgets that need them

Prefer

- watch or select inside build()
- create: inside a provider
- Unmodifiable getters
- Providers above MaterialApp or the route

Complete example: Todo app

todo_model.dart

```
class TodoModel extends ChangeNotifier {
  final _todos = <Todo>[];
  List<Todo> get todos =>
      List.unmodifiable(_todos);
  int get remaining =>
      _todos.where((t) => !t.done).length;

  void add(String title) {
    _todos.add(Todo(title));
    notifyListeners();
  }

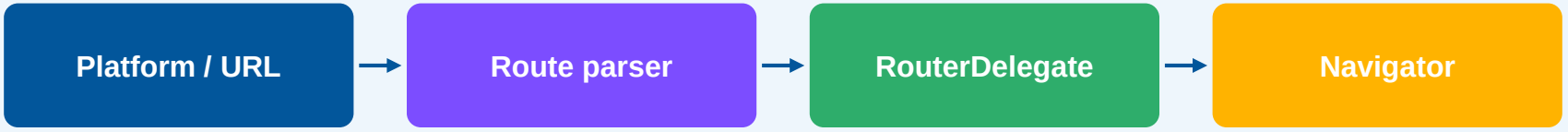
  void toggle(Todo t) {
    t.done = !t.done;
    notifyListeners();
  }
}
```

todo_page.dart

```
class TodoPage extends StatelessWidget {
  const TodoPage({super.key});

  @override
  Widget build(BuildContext context) {
    final model = context.watch<TodoModel>();
    return Scaffold(
      appBar: AppBar(title: Text(
          model.remaining.toString() + ' left')),
      body: ListView(children: [
        for (final t in model.todos)
          CheckboxListTile(
            title: Text(t.title),
            value: t.done,
            onChanged: (_) => context
                .read<TodoModel>().toggle(t),
          ),
      ]),
    );
  }
}
```

Navigator 2.0: declarative routing



- Navigator 1.0 is imperative: you call push and pop
- Navigator 2.0 is declarative: the page stack is derived from state and the URL
- The raw Router API is powerful but verbose, so most apps use go_router, which wraps it

Why go_router?

Navigator 1.0	go_router
• Imperative push and pop • Named routes with limited parameters • Deep links and web URLs need extra work • Back button and URL handled by hand	• Routes declared in one tree by path • Path and query parameters built in • Deep links and web URLs map to routes • Redirects, shell routes, transitions; maintained by the Flutter team

Setting up go_router

- flutter pub add go_router
- Create the GoRouter once, outside build()
- Use MaterialApp.router with routerConfig
- Each GoRoute has a path and a builder

router.dart

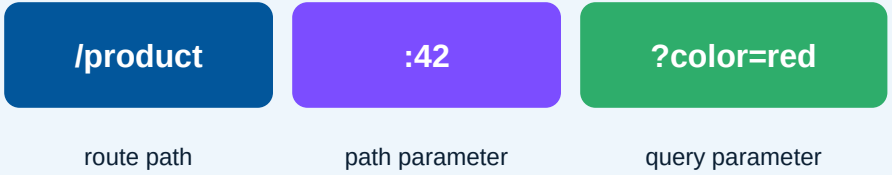
```dart
import 'package:go_router/go_router.dart';

final _router = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(
      path: '/',
      name: 'home',
      builder: (context, state) => const HomePage(),
    ),
  ],
);

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) =>
      MaterialApp.router(routerConfig: _router);
}
```

Path and query parameters



- Path params: state.pathParameters['id']
- Query params: state.uri.queryParameters
- Prefer goNamed to avoid typos in URLs

product_route.dart

```
GoRoute(
  path: '/product/:id',
  name: 'product',
  builder: (context, state) {
    final id = state.pathParameters['id']!;
    final color = state.uri.queryParameters['color'];
    return ProductPage(id: id, color: color);
  },
)

context.go('/product/42?color=red');

context.goNamed('product',
  pathParameters: {'id': '42'},
  queryParameters: {'color': 'red'});
```

go(), push() and pop()

G go()

Replaces the stack to match the location, like a URL

P push()

Adds a page on top; supports results

B pop()

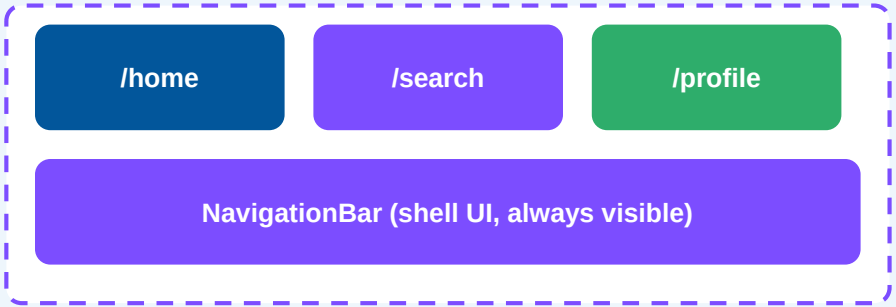
Removes the top page, optionally with a value

navigation_calls.dart

```
context.go('/profile');           // replace stack
context.push('/profile/edit');    // add on top
context.pop();                    // go back
context.pop('saved');             // return a result

final ok = await context.push<bool>('/confirm');  // await a result
context.pushReplacement('/login');                // swap current page
context.goNamed('settings');                      // by route name
```

Shell routes and bottom navigation

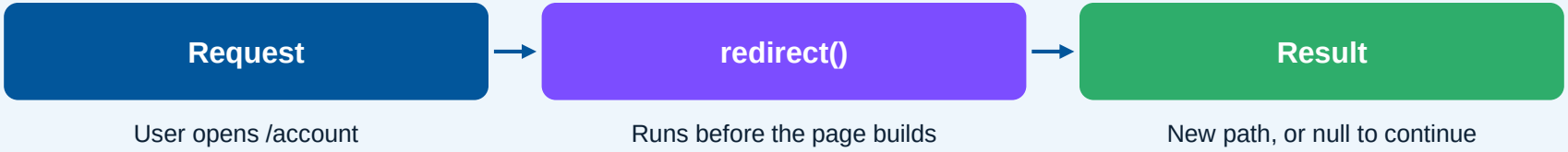


- The shell keeps the nav bar while pages change
- indexedStack keeps each tab's state and stack
- goBranch(i) switches tabs

shell_route.dart

```
StatefulShellRoute.indexedStack(
  builder: (context, state, shell) =>
      ScaffoldWithNavBar(navigationShell: shell),
  branches: [
    StatefulShellBranch(routes: [
      GoRoute(path: '/home', builder: (_, __) => const HomeTab())]),
    StatefulShellBranch(routes: [
      GoRoute(path: '/search', builder: (_, __) => const SearchTab())]),
  ],
)
// NavigationBar: onDestinationSelected: (i) => shell.goBranch(i)
```

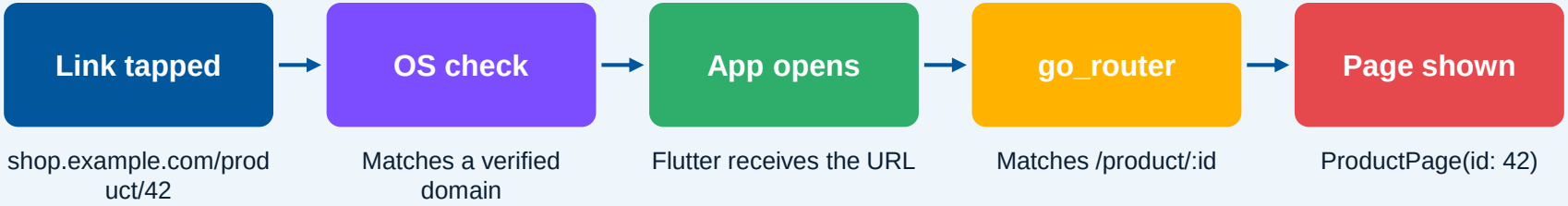
Redirects and auth guards



auth_router.dart

```
GoRouter(
  refreshListenable: authModel, // re-run on login or logout
  redirect: (context, state) {
    final loggedIn = authModel.isLoggedIn;
    final atLogin = state.matchedLocation == '/login';
    if (!loggedIn && !atLogin) return '/login';
    if (loggedIn && atLogin) return '/';
    return null; // no redirect
  },
  routes: appRoutes,
)
```

How a deep link reaches a page



U URI scheme

myapp://product/42. Simple, but any app can claim it.

A App Links / Universal Links

https links verified against your domain. Recommended.

W Web URLs

Path URLs map directly to routes in Flutter web.

Platform setup

AndroidManifest.xml

```
<intent-filter android:autoVerify="true">
  <action android:name="android.intent.action.VIEW"/>
  <category android:name="android.intent.category.DEFAULT"/>
  <category android:name="android.intent.category.BROWSABLE"/>
  <data android:scheme="https" android:host="shop.example.com"/>
</intent-filter>
<!-- recent Flutter versions handle the rest automatically -->
```

iOS, hosting and web

```
<!-- iOS Runner.entitlements: Associated Domains -->
<string>applinks:shop.example.com</string>

# Host on your domain
#   /.well-known/assetlinks.json              (Android)
#   /.well-known/apple-app-site-association    (iOS)

// Web: clean URLs without #
void main() { usePathUrlStrategy(); runApp(const MyApp()); }
```

Test deep links, then ship

terminal

```
# Android device or emulator
adb shell am start -a android.intent.action.VIEW \
  -d "https://shop.example.com/product/42"

# iOS simulator
xcrun simctl openurl booted \
  "https://shop.example.com/product/42"

# Check that the file is reachable
shop.example.com/.well-known/assetlinks.json
```

- setState: keep it local and small
- Provider: share state with watch, read, select
- go_router: the URL drives your pages